

Spectral clustering

팜티투짱-202139411(컴퓨터공학과)

Contents

1. Concept and Foundation
2. Algorithm and Walkthrough
3. Pros and Cons
4. Python Example
5. References



1. Concept and Foundation

1.1. What is spectral clustering?

1.2. Why use spectral clustering?

1.1.What is spectral clustering?

- **Spectral Clustering** is a graph-based clustering algorithm that groups data points according to their connectivity.

It constructs a similarity graph where each data point is a node, and the edge weights represent pairwise similarities.

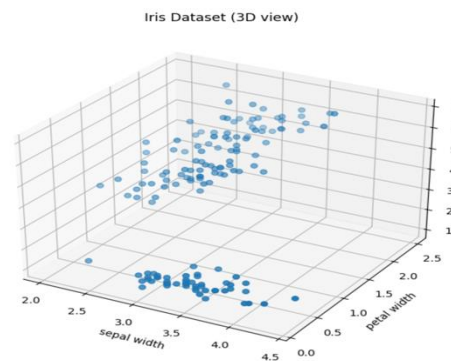
- The algorithm then uses the eigenvalues and eigenvectors of the graph Laplacian to project the data into a low-dimensional space where clusters become easily separable.

1.2. Why use spectral clustering?

-Transform the original data into a low-dimensional space where clusters are more easily separable, facilitating effective clustering.

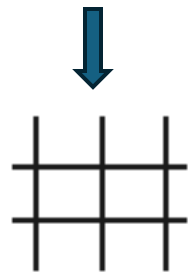
	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2
...	...	...	...	...
145	8.7	3.0	5.2	2.3
146	8.3	2.5	5.0	1.9
147	8.5	3.0	5.2	2.0
148	8.2	3.4	5.4	2.3
149	5.9	3.0	5.1	1.8

150 rows x 4 columns



- Data used: iris dataset(150 sample, 4features)
- The process:

Data of n object and m features



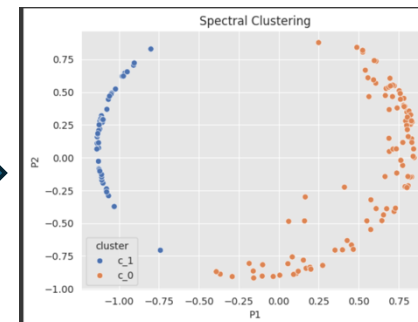
$n \times n$ weight matrix

Graph
Laplacian

	v_0	v_1
0	0.141421	0.0
1	0.141421	0.0
2	0.141421	0.0
3	0.141421	0.0
4	0.141421	0.0
...	...	...
145	0.000000	-0.1
146	0.000000	-0.1
147	0.000000	-0.1
148	0.000000	-0.1
149	0.000000	-0.1

[150 rows x 2 columns]

$n \times k$ matrix($k < n$)



input data

Similarity matrix

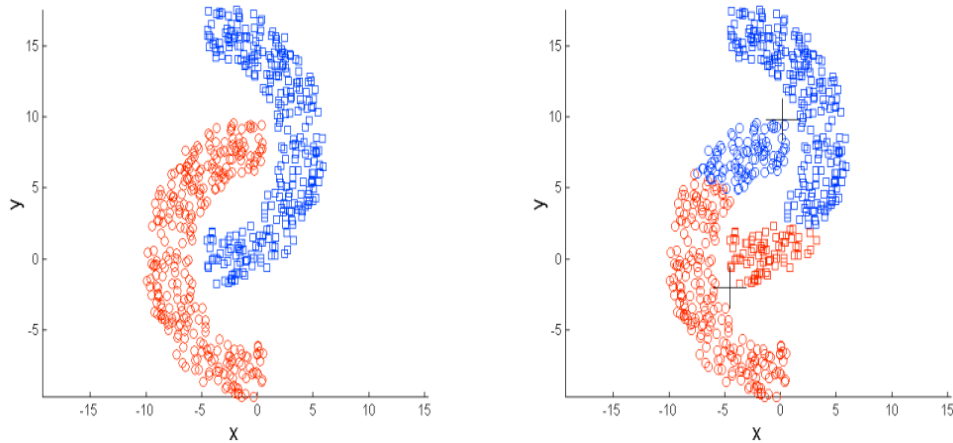
Laplacian

Eigen decomposition

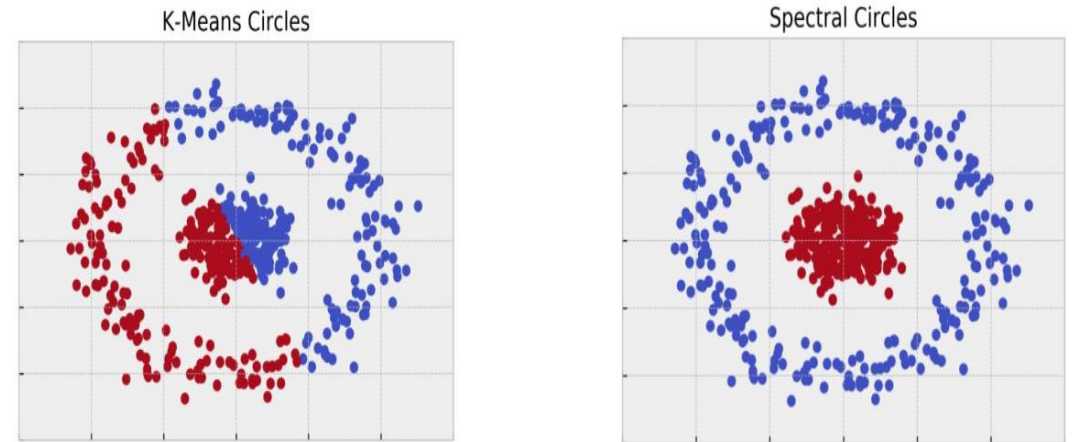
Cluster result

1.2. Why use spectral clustering?

-Spectral Clustering **handles complex-shaped clusters** that classical methods like k-means cannot separate



- Distance based method (like k-means) may not work well



Comparison between results of k-means clustering and spectral clustering

2. Algorithm and Walkthrough

2.1. Construct a similarity matrix S for the dataset with n samples, then derive the adjacency matrix A + degree matrix D

2.2. Build the **graph Laplacian** L using A and D

2.3. Compute the **eigenvalues and eigenvectors** of L

2.4. Select the k eigenvectors corresponding to the smallest eigenvalues and form an $n \times k$ **matrix U** .

2.5. Treat each row of U as a new data point and apply a **simple clustering algorithm** such as **K-means** to group these points into clusters.

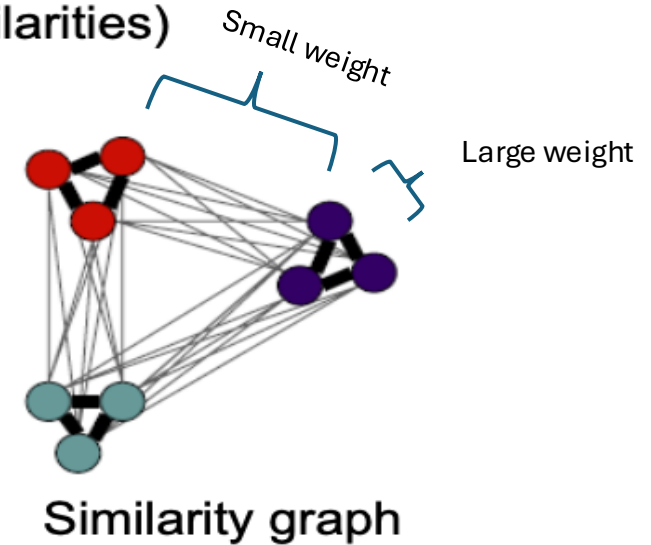
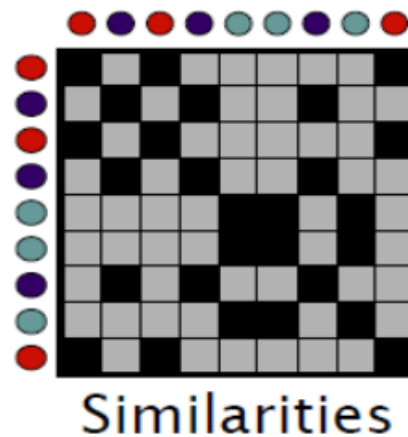
2.1. similarity matrix S

Similarity Graph: $G(V,E,W)$

V – Vertices (Data points)

E – Edge if similarity > 0

W - Edge weights (similarities)



- Edges within a group have large weights
- Edges across groups have small weights

2.1. similarity matrix S

	Gene 1	Gene 2
Cell 1	3	0.3
Cell 2	2.8	0.3
Cell 3	2.6	0.3
Cell 4	...	...
Cell 5	...	...
Cell 6	...	...

$$\text{Dist}_{12} = \sqrt{(3-2.8)^2 + (0.3-0.3)^2} = 0.2$$

$$\text{Dist}_{13} = \sqrt{(3-2.6)^2 + (0.3-0.3)^2} = 0.4$$

Distance matrix

	1	2	3	4	5
2	0.2				
3	0.4	0.2			
4	1	1	0.8		
5	0.9	1	1	0.2	
6	1	1	1	0.3	0.2

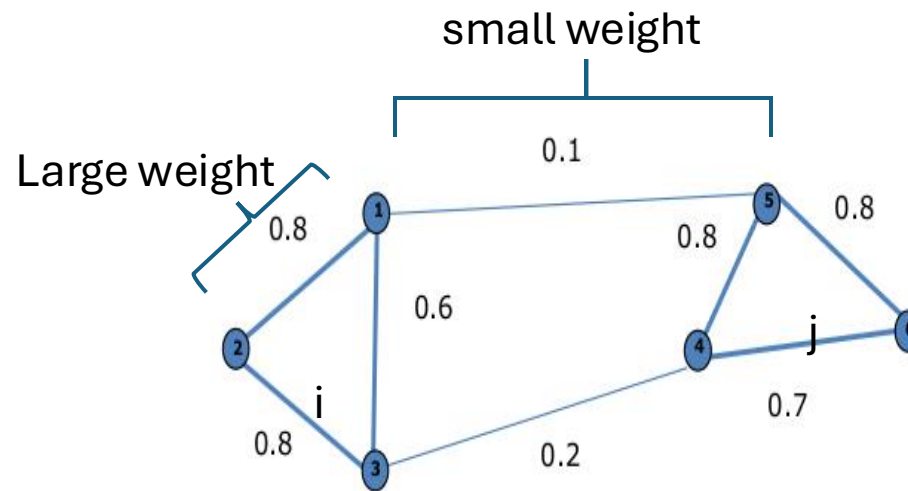
$$\text{Similarity} = 1 - \text{distance}/\text{max}(\text{distance})$$

	1	2	3	4	5
2	0.8				
3	0.6	0.8			
4	0	0	0.2		
5	0.1	0	0	0.8	
6	0	0	0	0.7	0.8

2.1. Adjacency matrix A

Similarity graph

	1	2	3	4	5
2	0.8				
3	0.6	0.8			
4	0	0	0.2		
5	0.1	0	0	0.8	
6	0	0	0	0.7	0.8



Adjacency matrix

	x_1	x_2	x_3	x_4	x_5	x_6
x_1	0	0.8	0.6	0	0.1	0
x_2	0.8	0	0.8	0	0	0
x_3	0.6	0.8	0	0.2	0	0
x_4	0	0	0.2	0	0.8	0.7
x_5	0.1	0	0	0.8	0	0.8
x_6	0	0	0	0.7	0.8	0

$$A_{ij} = \begin{cases} w_{ij} & \text{--> weight of edge between } i \text{ and } j \\ 0 & \text{--> if no edge in between } i \text{ and } j \end{cases}$$

$$A = [w_{ij}]$$

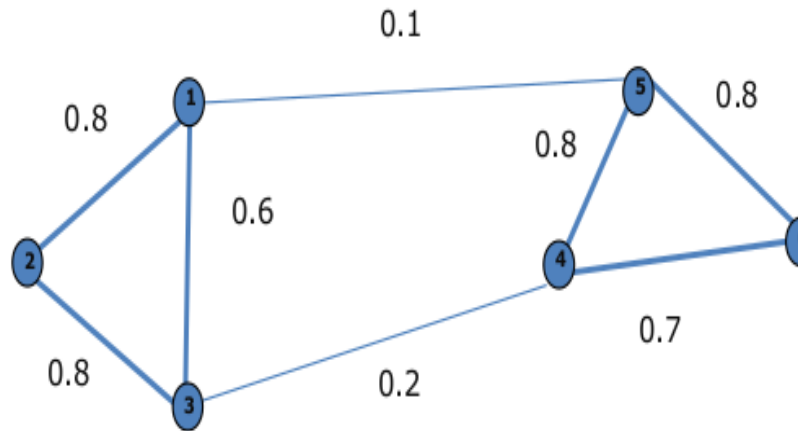
[td://blog](http://blog.td://blog)

2.1.Degree matrix(D)

- Degree: total weight of edges incident to vertex i . $d_i = \sum_{j=1}^n w_{ij}$

matrixAdjacency

	x_1	x_2	x_3	x_4	x_5	x_6
x_1	0	0.8	0.6	0	0.1	0
x_2	0.8	0	0.8	0	0	0
x_3	0.6	0.8	0	0.2	0	0
x_4	0	0	0.2	0	0.8	0.7
x_5	0.1	0	0	0.8	0	0.8
x_6	0	0	0	0.7	0.8	0



Degree matrix

	x_1	x_2	x_3	x_4	x_5	x_6
x_1	1.5	0	0	0	0	0
x_2	0	1.6	0	0	0	0
x_3	0	0	1.6	0	0	0
x_4	0	0	0	1.7	0	0
x_5	0	0	0	0	1.7	0
x_6	0	0	0	0	0	1.5

$$D_1 = 0.8 + 0.6 + 0.1 = 1.5$$

$$D_2 = 0.8 + 0.8 = 1.6$$

$$D_3 = 0.6 + 0.8 + 0.2 = 1.6$$

2.2. Un-normalized laplacian: $L = D - A$

L

D

	x_1	x_2	x_3	x_4	x_5	x_6
x_1	1.5	0	0	0	0	0
x_2	0	1.6	0	0	0	0
x_3	0	0	1.6	0	0	0
x_4	0	0	0	1.7	0	0
x_5	0	0	0	0	1.7	0
x_6	0	0	0	0	0	1.5

A

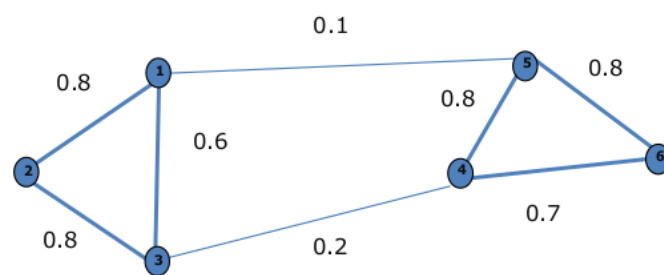
-

	x_1	x_2	x_3	x_4	x_5	x_6
x_1	0	0.8	0.6	0	0.1	0
x_2	0.8	0	0.8	0	0	0
x_3	0.6	0.8	0	0.2	0	0
x_4	0	0	0.2	0	0.8	0.7
x_5	0.1	0	0	0.8	0	0.8
x_6	0	0	0	0.7	0.8	0

=

	x_1	x_2	x_3	x_4	x_5	x_6
x_1	1.5	-0.8	-0.6	0	-0.1	0
x_2	-0.8	1.6	-0.8	0	0	0
x_3	-0.6	-0.8	1.6	-0.2	0	0
x_4	0	0	-0.2	1.7	-0.8	-0.7
x_5	-0.1	0	0	-0.8	1.7	-0.8
x_6	0	0	0	-0.7	-0.8	1.5

2.2. Graph Laplacian



	x_1	x_2	x_3	x_4	x_5	x_6
x_1	1.5	-0.8	-0.6	0	-0.1	0
x_2	-0.8	1.6	-0.8	0	0	0
x_3	-0.6	-0.8	1.6	-0.2	0	0
x_4	0	0	-0.2	1.7	-0.8	-0.7
x_5	-0.1	0	0	-0.8	1.7	-0.8
x_6	0	0	0	-0.7	-0.8	1.5

2.3. Compute the eigenvalues and eigenvectors of L

The unnormalized graph Laplacian matrix is defined as

$$L = D - W.$$

An overview over many of its properties can be found in Mohar (1991, 1997). The following proposition summarizes the most important facts needed for spectral clustering.

Proposition 1 (Properties of L) *The matrix L satisfies the following properties:*

1. *For every vector $f \in \mathbb{R}^n$ we have*

$$f'Lf = \frac{1}{2} \sum_{i,j=1}^n w_{ij}(f_i - f_j)^2.$$

2. *L is symmetric and positive semi-definite.*

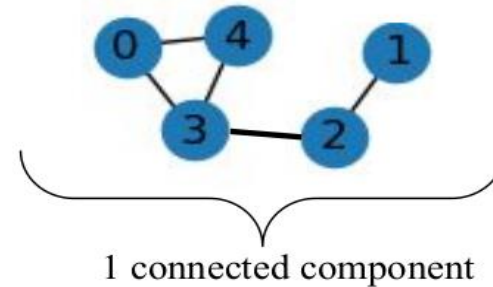
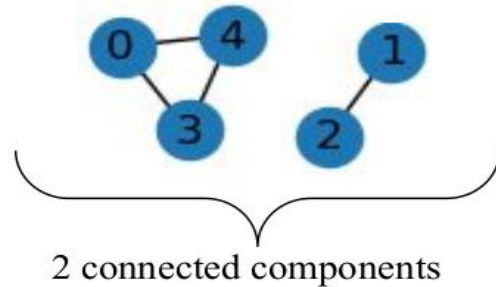
3. *The smallest eigenvalue of L is 0, the corresponding eigenvector is the constant one vector $\mathbb{1}$.*

4. *L has n non-negative, real-valued eigenvalues $0 = \lambda_1 \leq \lambda_2 \leq \dots \leq \lambda_n$.*

Zero eigenvalues and the connected components

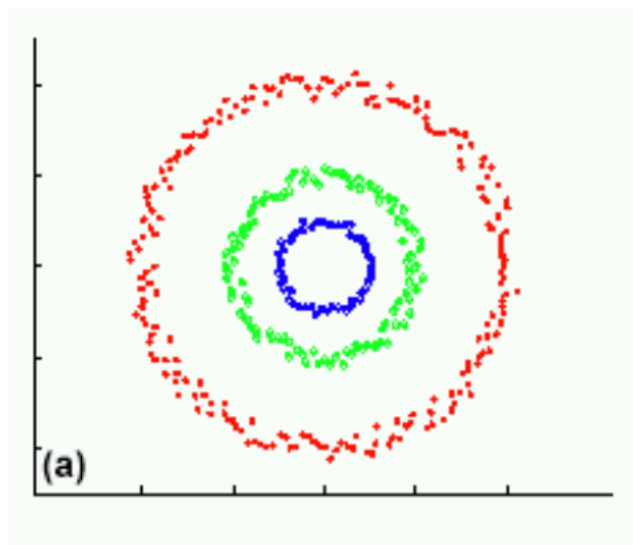
Proposition 2: Let G be an undirected graph with non-negative weight matrix W . Then :

- The multiplicity k of the eigenvalue 0 of L equals the number of connected components $C_1, \dots, C_k$ in the graph.

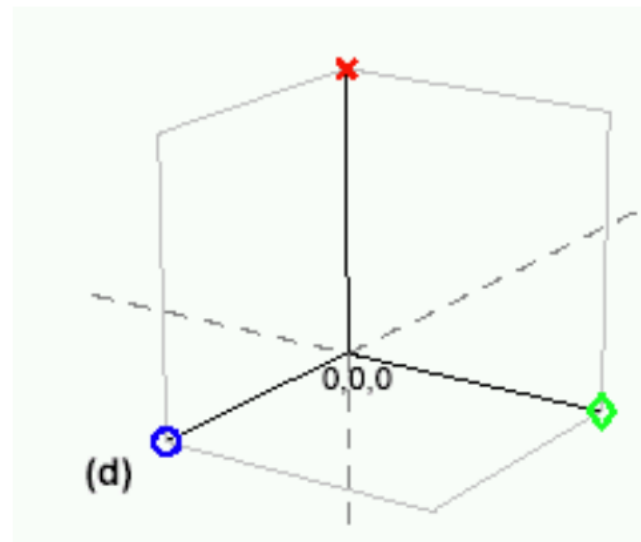


- The eigenspace of eigenvalue 0 is spanned by k **indicator vectors** which map the n nodes to the k clusters.

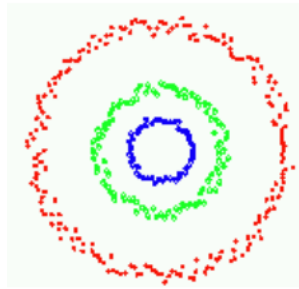
Original data



Projected data



If graph is connected $\rightarrow$ 1 eigenvector all 1s; if disconnected $\rightarrow$ k eigenvectors constant per cluster



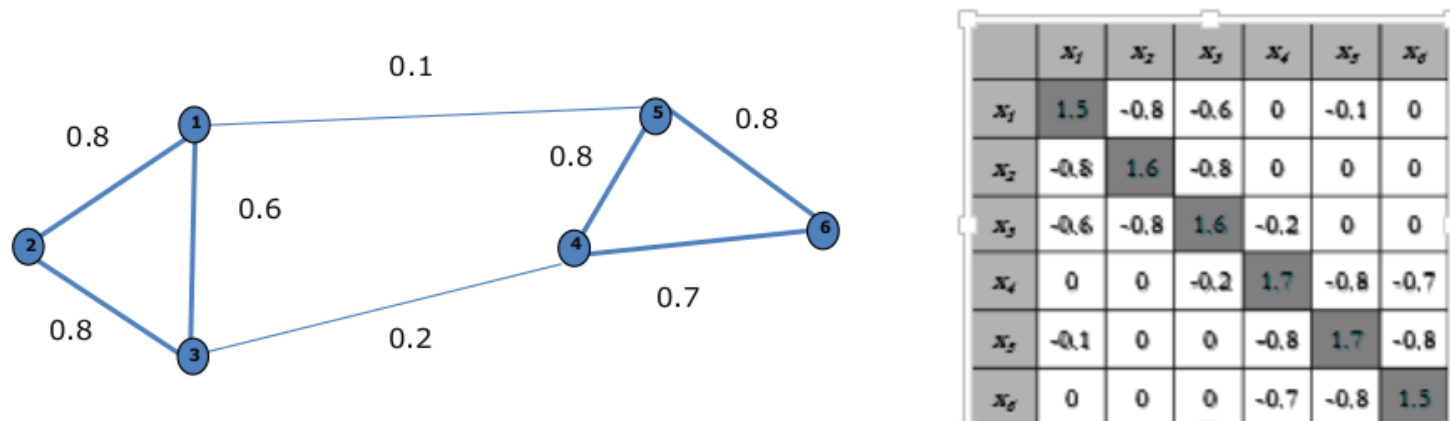
OR



$$L = \begin{bmatrix} L_1 & & & \\ & L_2 & & \\ & & \ddots & \\ & & & 0 \\ & & & & \ddots \\ 0 & & & & & L_3 \\ & & & & & & \ddots \end{bmatrix}$$

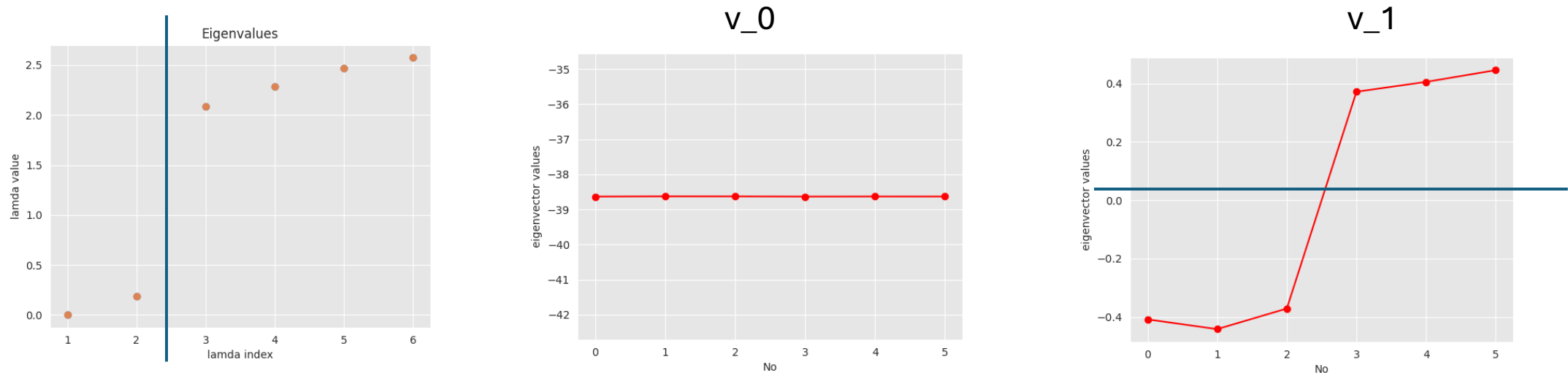
$$\begin{bmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix} \quad \begin{bmatrix} 0 \\ \vdots \\ 0 \\ 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix} \quad \begin{bmatrix} 0 \\ \vdots \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

First three eigenvectors



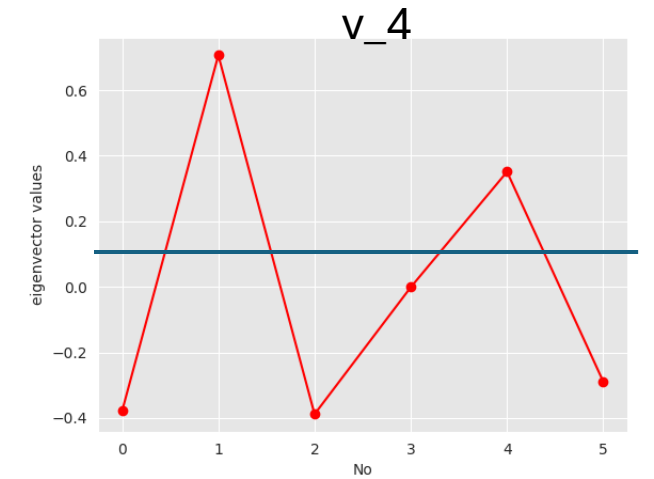
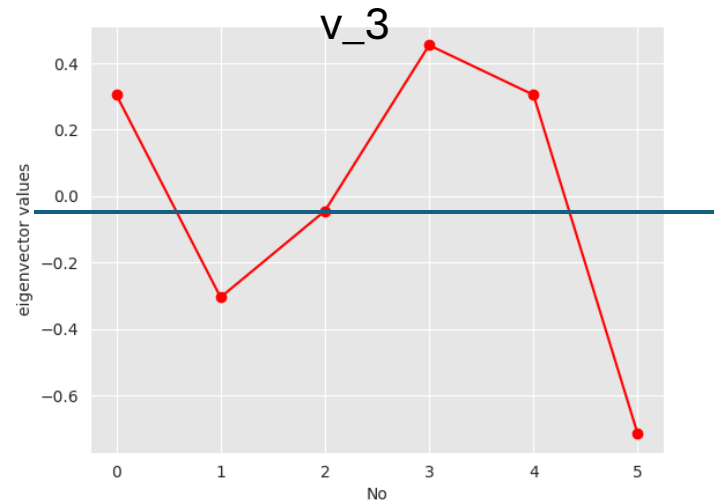
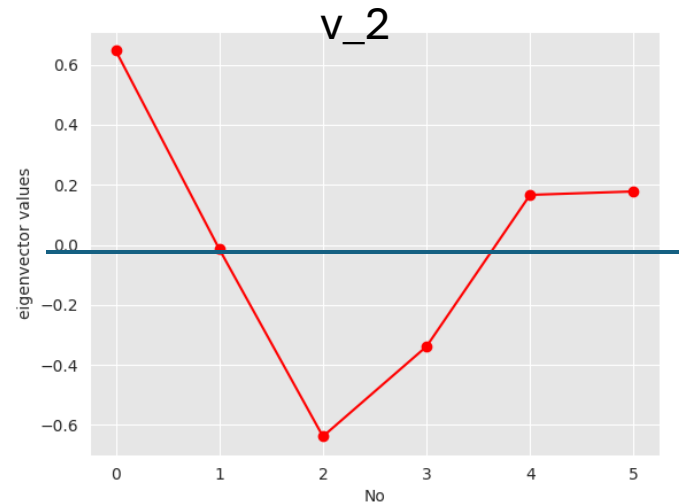
Connected graph $\rightarrow$ one zero eigenvalue $\rightarrow$ one cluster ($C = [1 \ 1 \ 1 \ 1 \ 1 \ 1]^T$)

2.4. Select the k eigenvectors corresponding to the smallest eigenvalues and form an $n \times k$ matrix U .

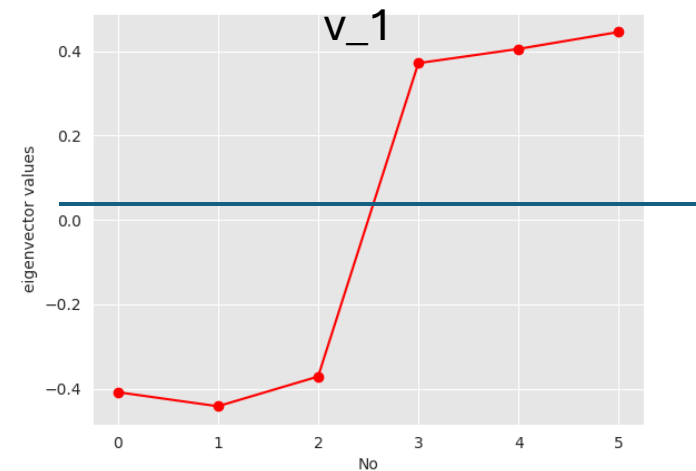


Braun, and Buhmann, 2004; Ben-David, von Luxburg, and Pál, 2006). Of course all those methods can also be used for spectral clustering. Additionally, one tool which is particularly designed for spectral clustering is the eigengap heuristic, which can be used for all three graph Laplacians. Here the goal is to choose the number k such that all eigenvalues $\lambda_1, \dots, \lambda_k$ are very small, but λ_{k+1} is relatively large. There are several justifications for this procedure. The first one is based on perturbation theory, where we observe that in the ideal case of k completely disconnected clusters, the eigenvalue 0 has multiplicity k , and then there is a gap to the $(k + 1)$ th eigenvalue $\lambda_{k+1} > 0$. Other explanations can be given by spectral graph theory. Here, many geometric invariants of the graph can be expressed or bounded with the help of the first eigenvalues of the graph Laplacian. In particular, the sizes of cuts are closely related to the size of the first eigenvalues. For more details on this topic we refer to Bolla (1991), Mohar (1997) and Chung (1997).

Choose k, case: fully connected graph

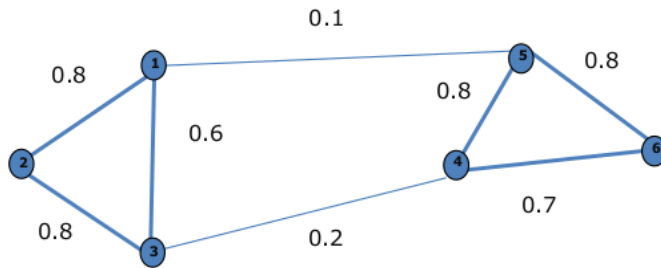


k=2 best

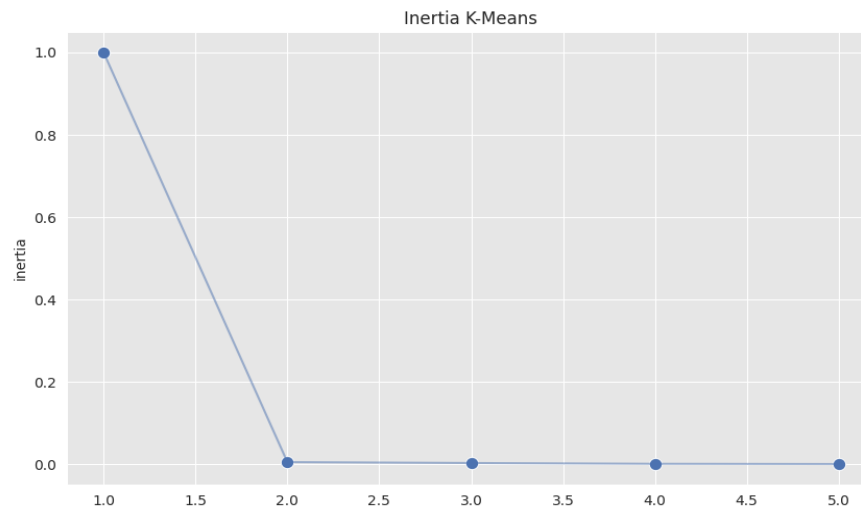


2.5. Treat each row of U as a new data point, and apply a simple clustering algorithm such as K-means to group these points into clusters.

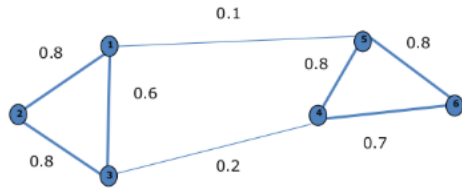
Matrix U , $k=2$



	v_0	v_1	No
0	-0.408248	-0.408401	0
1	-0.408248	-0.441825	1
2	-0.408248	-0.371319	2
3	-0.408248	0.371334	3
4	-0.408248	0.405048	4
5	-0.408248	0.445163	5



Graph Laplacians: second smallest eigenvalue - Bipartition (k=2)



$$\lambda_2 = 0.189$$

	x_1	x_2	x_3	x_4	x_5	x_6
x_1	1.5	-0.8	-0.6	0	-0.1	0
x_2	-0.8	1.6	-0.8	0	0	0
x_3	-0.6	-0.8	1.6	-0.2	0	0
x_4	0	0	-0.2	1.7	-0.8	-0.7
x_5	-0.1	0	0	-0.8	1.7	-0.8
x_6	0	0	0	-0.7	-0.8	1.5

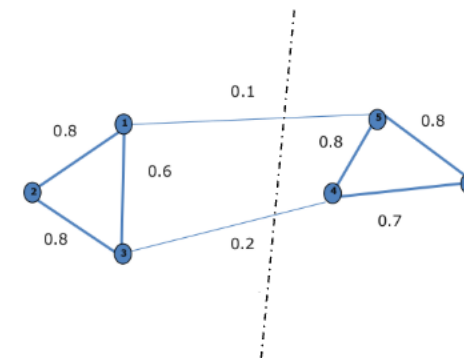
laplacian

- To find the bipartition, we take the second eigenvector of the Laplacian, $\mathbf{v}_2$, corresponding to λ_2 , the algebraic connectivity of G.
- The smaller λ_2 , the better quality of the partitioning
- For each node i in G, assign it the value $\mathbf{v}_2(i)$ e.g., $\mathbf{v}_2(1) = 0.41$ in G
- To find clusters C_1 and C_2 , assign nodes with $\mathbf{v}_2(i) > 0$ to C_1 and $\mathbf{v}_2(i) < 0$ to C_2 .

$$C_1 = \{i \mid \mathbf{v}_2(i) > 0\},$$

$$C_2 = \{i \mid \mathbf{v}_2(i) < 0\}$$

	$\mathbf{v}_2$
1	-0.408
2	-0.442
3	-0.371
4	0.371
5	0.405
6	0.445

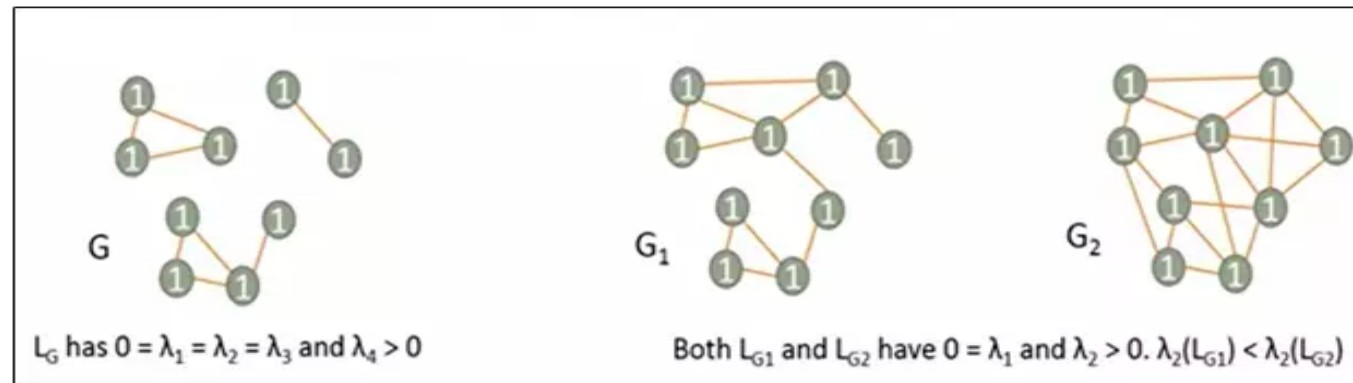


Graph Laplacians: second smallest eigenvalue

- If 0 is the eigenvalue of L with k different eigenvectors, i.e., $0 = \lambda_1 = \lambda_2 \dots = \lambda_k$, then the graph G has k connected components
- If the graph is connected, $\lambda_2 > 0$ and λ_2 is the algebraic connectivity of G
- The greater λ_2 is, the more connected G is

Graph Laplacians: second smallest eigenvalue

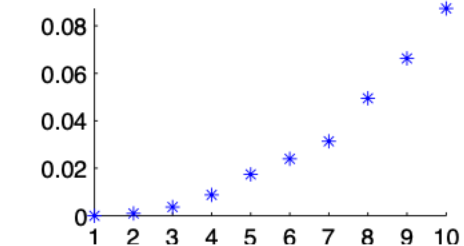
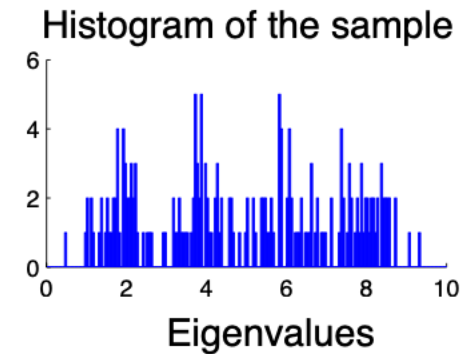
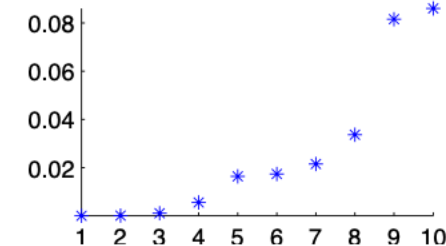
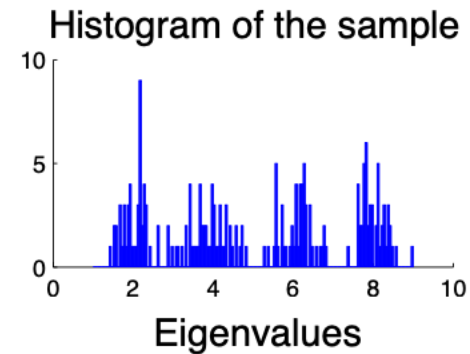
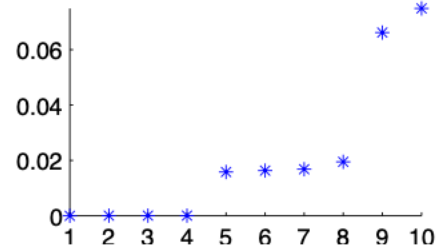
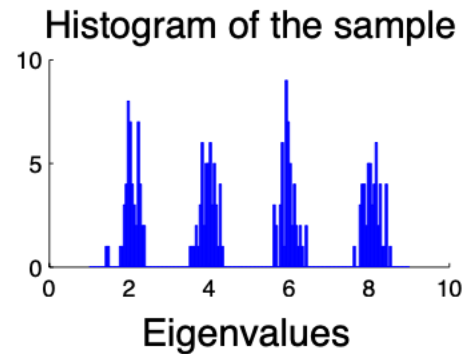
- Nếu 0 là giá trị riêng của L ($\lambda_1 = \lambda_2 = \dots = \lambda_k = 0$) với k vector riêng thì đồ thị G có k kết nối thành phần
- Nếu đồ thị đã được kết nối, $\lambda_2 > 0$ và λ_2 là kết nối đại số của G , λ_2 càng lớn thì G càng có nhiều kết nối.



Dễ thấy rằng L_G có $\lambda_1 = \lambda_2 = \lambda_3 = 0$ và $\lambda_4 > 0$ nên G có 3 đường kết nối. L_{G_1} và L_{G_2} có $\lambda_1 = 0$ nên 2 đồ thị G_1 và G_2 có một đường kết nối.
Lại có $\lambda_2(L_{G_1}) < \lambda_2(L_{G_2})$ nên G_2 có nhiều đường kết nối hơn.

Disadvantage

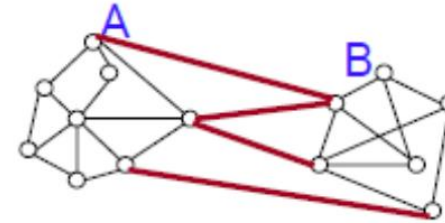
Choice of number of clusters k Most stable clustering is usually given by the value of k that maximizes the eigengap (difference between consecutive eigenvalues)



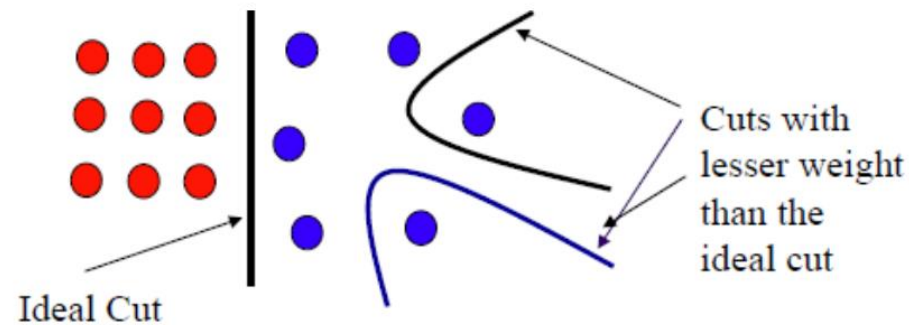
What makes a good graph cut?

Min-cut: Partition graph into two sets A and B such that weight of edges connecting vertices in A to vertices in B is minimum.

$$\text{cut}(A, B) := \sum_{i \in A, j \in B} w_{ij}$$

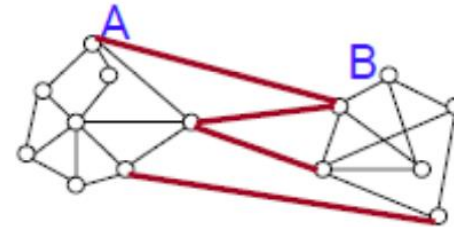


- Easy to solve $O(VE)$ algorithm
- Not satisfactory partition – often isolates vertices



Partition graph into two sets A and B such that weight of edges connecting vertices in A to vertices in B is minimum & size of A and B are very similar.

$$\text{cut}(A, B) := \sum_{i \in A, j \in B} w_{ij}$$



Normalized cut:

$$\text{Ncut}(A, B) := \text{cut}(A, B) \left(\frac{1}{\text{vol}(A)} + \frac{1}{\text{vol}(B)} \right)$$

$$\text{vol}(A) = \sum_{i \in A} d_i$$

But NP-hard to solve!!

Spectral clustering is a relaxation of these.

Other spectral clustering algorithms(참조)

- Normalized Graph Laplacian

$$L_{sym} := D^{-\frac{1}{2}}LD^{-\frac{1}{2}} = I - D^{-\frac{1}{2}}WD^{-\frac{1}{2}}$$

$$L_{rw} := D^{-1}L = I - D^{-1}W$$

We denote the first matrix by L_{sym} as it is a symmetric matrix, and the second one by L_{rw} as it is closely related to a random walk.

Normalized spectral clustering according to Shi and Malik (2000)

Input: Similarity matrix $S \in \mathbb{R}^{n \times n}$, number k of clusters to construct.

- Construct a similarity graph by one of the ways described in Section 2. Let W be its weighted adjacency matrix.
- Compute the unnormalized Laplacian L .
- Compute the first k generalized eigenvectors $u_1, \dots, u_k$ of the generalized eigenproblem $Lu = \lambda Du$.
- Let $U \in \mathbb{R}^{n \times k}$ be the matrix containing the vectors $u_1, \dots, u_k$ as columns.
- For $i = 1, \dots, n$, let $y_i \in \mathbb{R}^k$ be the vector corresponding to the i -th row of U .
- Cluster the points $(y_i)_{i=1, \dots, n}$ in $\mathbb{R}^k$ with the k -means algorithm into clusters $C_1, \dots, C_k$.

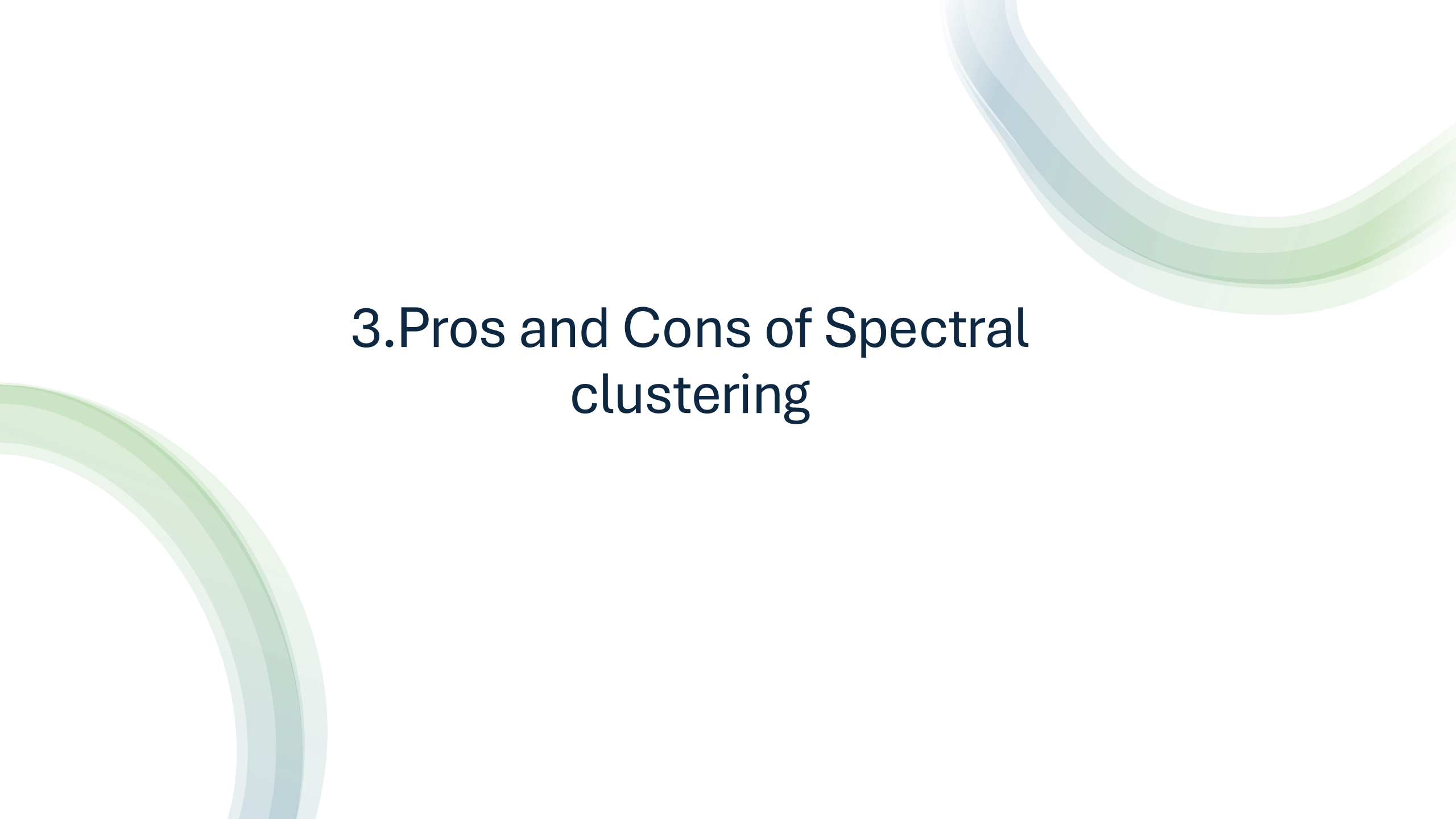
Output: Clusters $A_1, \dots, A_k$ with $A_i = \{j \mid y_j \in C_i\}$.

Normalized spectral clustering according to Ng, Jordan, and Weiss (2002)

Input: Similarity matrix $S \in \mathbb{R}^{n \times n}$, number k of clusters to construct.

- Construct a similarity graph by one of the ways described in Section 2. Let W be its weighted adjacency matrix.
- Compute the normalized Laplacian L_{sym} .
- Compute the first k eigenvectors $u_1, \dots, u_k$ of L_{sym} .
- Let $U \in \mathbb{R}^{n \times k}$ be the matrix containing the vectors $u_1, \dots, u_k$ as columns.
- Form the matrix $T \in \mathbb{R}^{n \times k}$ from U by normalizing the rows to norm 1, that is set $t_{ij} = u_{ij} / (\sum_k u_{ik}^2)^{1/2}$.
- For $i = 1, \dots, n$, let $y_i \in \mathbb{R}^k$ be the vector corresponding to the i -th row of T .
- Cluster the points $(y_i)_{i=1, \dots, n}$ with the k -means algorithm into clusters $C_1, \dots, C_k$.

Output: Clusters $A_1, \dots, A_k$ with $A_i = \{j \mid y_j \in C_i\}$.

The slide features decorative curved lines in the corners. In the top-right corner, there is a thick, multi-layered arc transitioning from light blue to light green. In the bottom-left corner, there is a similar thick, multi-layered arc transitioning from light green to light blue. The central text is positioned between these decorative elements.

3. Pros and Cons of Spectral clustering

Pros

- Works for **non-convex / complex clusters**
- **Simple & efficient** (linear algebra only)
- **No local minima** problem
- **Scalable** with sparse graphs

Cons

- **Sensitive** to similarity graph & parameters
- **Unstable** with noise or tuning changes
- **Unnormalized** version may fail to converge
- Gives only **cluster labels**, no model

The slide features decorative curved lines in the corners. In the top-right corner, there is a thick, multi-layered arc transitioning from light blue to light green. In the bottom-left corner, there is a similar thick, multi-layered arc transitioning from light green to light blue.

4. Python Example

```
import pandas as pd
from matplotlib import pyplot
from pandas import DataFrame
import seaborn as sns
sns.set_style('darkgrid', {'axes.facecolor': '.9'})
sns.set_palette(palette='deep')
sns_c = sns.color_palette(palette='deep')
import numpy as np
from pandas.plotting import register_matplotlib_converters
register_matplotlib_converters()
from sklearn.cluster import KMeans
from sklearn.cluster import SpectralClustering
from sklearn.preprocessing import StandardScaler, normalize
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score
```

```
## Create Laplacian matrix
from sklearn.neighbors import kneighbors_graph
from scipy import sparse
## input data
raw_df = pd.read_csv('/content/Iris.csv')
raw_df
# numpy array
array = raw_df.to_numpy()
array
```

```
## Create Laplacian matrix
def generate_graph_laplacian(array, nn):
    """Generate graph Laplacian from data."""
    # Adjacency Matrix.
    connectivity = kneighbors_graph(X=array, n_neighbors=nn, mode='connectivity')
    adjacency_matrix_w = (1/2)*(connectivity + connectivity.T)
    # Graph Laplacian, normed=normalization.
    graph_laplacian_s = sparse.csgraph.laplacian(csgraph=adjacency_matrix_w, normed=False)
    graph_laplacian = graph_laplacian_s.toarray()
    return graph_laplacian
graph_laplacian = generate_graph_laplacian(array=array, nn=8)
print("Matrix Laplacian:")
print(graph_laplacian)
```

```
## Compute eigenvectors and eigenvalues
from scipy import linalg
def compute_spectrum_graph_laplacian(graph_laplacian):
    eigenvals, eigenvcts = linalg.eig(graph_laplacian)
    eigenvals = np.real(eigenvals)
    eigenvcts = np.real(eigenvcts)
    return eigenvals, eigenvcts
eigenvals, eigenvcts = compute_spectrum_graph_laplacian(graph_laplacian)
print("Eigenvalues Graph Laplacian:")
print(eigenvals)
print("Eigenvectors Graph Laplacian:")
print(eigenvcts)
```

```
# Graph visualizes eigenvalues ~0.

# choose the first 10 indices from the eigenvalue graph above,

# we can clearly see which eigenvalues ~ 0 of the graph Laplacian.

index_lim = 10

fig, ax = pyplot.subplots(figsize=(10, 6))

sns.scatterplot(x=range(1, eigenvals_sorted_indices[: index_lim].size + 1), y=eigenvals_sorted[: index_lim], s=80, ax=ax)

sns.lineplot(x=range(1, eigenvals_sorted_indices[: index_lim].size + 1), y=eigenvals_sorted[: index_lim], alpha=0.5, ax=ax)

ax.axvline(x=2, color=sns_c[3], label='zero eigenvalues', linestyle='--') # choose x

ax.set(title=f'Sorted Eigenvalues Graph Laplacian (First {index_lim})', xlabel='index', ylabel=r'$\lambda$')

pyplot.show()
```

```
# eigenvalues ~ 0
```

```
zero_eigenvals_index = np.argwhere(abs(eigenvals) < 1e-5)
```

```
eigenvals[zero_eigenvals_index]
```

```
# eigenvectors correspond to eigenvalues ~ 0
```

```
proj_df = pd.DataFrame(eigenvcts[:, zero_eigenvals_index.squeeze()])
```

```
proj_df.columns = ['v_' + str(c) for c in proj_df.columns]
```

```
print(proj_df)
```



```
def run_k_means(df, n_clusters):  
    """K-means clustering."""  
    k_means = KMeans(random_state=25, n_clusters=n_clusters)  
    k_means.fit(df)  
    cluster = k_means.predict(df)  
    return cluster  
  
cluster = run_k_means(proj_df, n_clusters=2) # n_clusters=k
```

```
# Normalize
```

```
X_normalized = normalize(X_scaled)
```

```
# Converting the numpy array into a pandas DataFrame
```

```
X_normalized = pd.DataFrame(X_normalized)
```

```
# Reducing the dimensions of the data
```

```
pca = PCA(n_components = 2)
```

```
df = pca.fit_transform(X_normalized)
```

```
df = pd.DataFrame(df)
```

```
df.columns = ['P1', 'P2']
```

```
df
```

```
df['cluster'] = ['c_' + str(c) for c in cluster]
```

```
df
```

```
fig, ax = pyplot.subplots()
```

```
sns.scatterplot(x='P1', y='P2', data = df, hue='cluster', ax=ax)
```

```
ax.set(title='Spectral Clustering')
```

```
pyplot.show()
```

summary

- Algorithms that cluster points using eigenvectors of matrices derived from the data
- Useful in hard non-convex clustering problems
- Obtain data representation in the low-dimensional space that can be easily clustered
- Variety of methods that use eigenvectors of unnormalized or normalized Laplacian, differ in how to derive clusters from eigenvectors, k-way vs repeated 2-way
- Empirically very successful

5. References

- A Tutorial on Spectral Clustering. Ulrike von Luxburg (2007)
- https://www.cs.cmu.edu/~aarti/Class/10701/slides/Lecture21_2.pdf
- <https://www.youtube.com/watch?v=zLuVrqIYKyg&list=PLHXZdjnADb1Q6oI2UEuPMnXpQY-KzCmBy&index=2>
- <https://www.youtube.com/watch?v=zkgm0i77jQ8>
- <https://viblo.asia/p/tim-hieu-thuat-toan-phan-cum-spectral-clustering-RQqKLBp0I7z>
- Github repository: Spectral clustering(code+ slides)
https://github.com/ptttrang1211/spectral_clustering_ML_2025_2.git



Thank
You!